

Terraform

- about Terraform:

1. infrastructure as a code = IAAC
2. define and maintain infrastructure state
3. compare Ansible, chef, puppet
4. Terraform + ansible to achieve automation on provision + configuration
5. <https://github.com/terraform-aws-modules>
6. <https://registry.terraform.io/modules/terraform-aws-modules>

- about enable and configure Terraform

1. choco install aws cli
2. choco install terraform
3. AWS user with access key + secret access key
4. AWS configure for access key + secret access key
5. terraform version

- E1: 1st instance

1. create key pair in AWS (key name)
2. create security group in AWS (SG ID)
3. AMI id, region, AZ, instance type
4. save .tf file
5. Terraform init, validate, fmt, plan, apply

```
1  provider "aws" {
2    region = "ap-southeast-2"
3  }
4
5  resource "aws_instance" "aws_instance_NM" {
6    ami           = "ami-0f5f922f781854672"
7    key_name      = "terraform-new-key1"
8    availability_zone = "ap-southeast-2a"
9    instance_type  = "t2.micro"
10   vpc_security_group_ids = ["sg-0861a6efb1ea9ba42"]
11   tags = {
12     Name = "T-N-T"
13   }
14 }
```

6. make changes to .tf file by add tags, Terraform init, validate, fmt, plan, apply
7. Terraform destroy

- E2: VAR

reuse code by change variables values

1. define AZ, tags, key, AMI in var
2. save providers, vars, instance.tf file
3. var. in instance.tf
4. Terraform init, validate, fmt, plan, apply
5. if change **key** in .tf, terraform will destroy the previous and add a new instance

- E3 with Ansible dynamic Inventory

1. install ansible on a linux server as ansible master

- yum install awscli
- yum install python3 -y && yum -y install python3-pip
- pip3 install boto3
- yum install python-boto3
- yum install epel-release -y
- yum install ansible -y

2. configure **dynamic Inventory plugin**

- export aws key, secret key + region

```
export AWS_ACCESS_KEY_ID=AKIARFI  
export AWS_SECRET_ACCESS_KEY=IZm  
DZg7  
export AWS_DEFAULT_REGION=ap-sou
```

- upload pem key to ansible folder
yum -y install lrzsz
rz to upload key
chmod 600 terraform-new-key1.pem
- vim /etc/ansible/ansible.cfg
to enable plugin + set inventory, remote user and key

```
[defaults]  
  
# some basic default values...  
inventory      = /opt/ansible/inventory/aws_ec2.yaml  
remote_user    = ec2-user  
private_key_file = /opt/ansible/inventory/terraform-new-key1.pem  
  
[inventory]  
# enable inventory plugins, default: 'host_list', 'script', 'auto'  
, 'yaml', 'ini', 'toml'  
#enable_plugins = host_list, virtualbox, yaml, constructed  
enable_plugins = aws_ec2
```

- vim aws_ec2.yaml
to create plugin, keyed_group

```
plugin: aws_ec2  
  
regions:  
- ap-southeast-2 # specify your AWS region  
  
keyed_groups:  
- key: tags.Name  
  prefix: tag_Name  
- key: placement.region  
  prefix: aws_region
```

- list ec2 and test connection

```
ansible all -m ping
ansible-inventory -i /opt/ansible/inventory/aws_ec2.yaml --list
```

- create playbook for a group of ec2

The primary use case of AWS Ansible dynamic inventory is to execute Ansible playbooks or ad-hoc commands against a single or group of categorized or grouped instances based on tags, regions, or other ec2 parameters.

You can group instances using tags, instances type, instance names, custom filters, and more. Take a look at all supported filters and keyed groups from https://docs.ansible.com/ansible/latest/collections/amazon/aws/aws_ec2_inventory.html

```
[root@localhost inventory]# cat pb1.yaml
---
- hosts: aws_region_ap_southeast_2
  become: true

  tasks:
    - name: Update system packages
      package:
        name: '*'
        state: latest
      become: yes
    - name: Install Apache (httpd)
      package:
        name: httpd
        state: present
      become: yes

    - name: Start and enable Apache service
      service:
        name: httpd
        state: started
        enabled: yes
      become: yes
```

to list and output defined dynamic group:

```
ansible-inventory --graph
```

```
[root@localhost inventory]# ansible-inventory --graph
@all:
  |--@aws_ec2:
  |   |--ec2-3-26-35-88.ap-southeast-2.compute.amazonaws.com
  |   |--ec2-54-252-248-22.ap-southeast-2.compute.amazonaws.com
  |--@aws_region_ap_southeast_2:
  |   |--ec2-3-26-35-88.ap-southeast-2.compute.amazonaws.com
  |   |--ec2-54-252-248-22.ap-southeast-2.compute.amazonaws.com
  |--@tag_Name_T_N_T_e2:
  |   |--ec2-54-252-248-22.ap-southeast-2.compute.amazonaws.com
  |--@tag_Name_T_N_T_e4:
  |   |--ec2-3-26-35-88.ap-southeast-2.compute.amazonaws.com
  |--@tag_Project_new_t_e2:
  |   |--ec2-54-252-248-22.ap-southeast-2.compute.amazonaws.com
  |--@tag_Project_new_t_e4:
  |   |--ec2-3-26-35-88.ap-southeast-2.compute.amazonaws.com
  |--@ungrouped:
```

- edit hosts and run playbook for above group
ansible-playbook -i aws_ec2.yaml pb1.yaml # for all ec2 in region

```
[root@localhost inventory]# vim pb1.yaml
```

```
---
- hosts: aws_region_ap_southeast_2
  become: true
```

ansible-playbook -i aws_ec2.yaml pb3.yaml # for group with tag
Project=new-t-e4

```
---
- hosts: tag_Project_new_t_e4
  become: true
```

- **E4: provisioner**
 1. create VPC subnets, IGW, NATGW, RT, RT association
 2. create SG
 3. create bastion in pub, backend server in private subnet
 4. use remote exec and file provisioner to configure bastion host for ansible ready
 - need "connection" to ssh to remote ec2

```
connection {
  type      = "ssh"
  user      = "ubuntu" # Replace with your EC2 user
  private_key = file("terraform-new-key1.pem") # Replace with the path to your private key
  host      = self.public_ip
}
```

- need file with source and destination to upload file to remote ec2

```
provisioner "file" {
  source      = "ansible.sh"
  destination = "/home/ubuntu/ansible.sh"
}

provisioner "file" {
  source      = "terraform-new-key1.pem"
  destination = "/home/ubuntu/terraform-new-key1.pem"
}
```

- need remote-exec to run command to

```
provisioner "remote-exec" {
  inline = [
    "cd /home/ubuntu/",
    "sudo chmod 600 terraform-new-key1.pem",
    "sudo chmod +x ansible.sh",
    "sudo ./ansible.sh",
    "sudo ansible-inventory -i /opt/ansible/inventory/aws_ec2.yaml --list",
    "sudo ansible-inventory --graph"
  ]
}
```

- use .sh to install and configure ec2 dynamic inventory

```
e3-easy > $ ansible.sh
1  #!/bin/bash
2
3  restart_config_file="/etc/needrestart/needrestart.conf"
4  sed -i "s/^#\$\$nrconf{restart} = 'i';/\$nrconf{restart} = 'a';/" "$restart_config_file"
5
6  sudo apt update -y
7  sudo apt -y install lrzsz
8  sudo apt install software-properties-common -y
9  sudo add-apt-repository --yes --update ppa:ansible/ansible -y
10 sudo apt install ansible -y
11 sudo apt-get install python3 -y
12 sudo apt-get install python3-pip -y
13 sudo apt-get install python-boto3 -y
14 sudo pip3 install boto3
15 sudo apt install awscli -y
16 sudo pip show boto3
17 sudo pip show botocore
18 sudo python3 --version
19 sudo ansible --version
20 sudo aws --version
21 |
22 cp /etc/ansible/ansible.cfg /etc/ansible/ansible.cfg.bk
23
24 echo [defaults] > /etc/ansible/ansible.cfg
25 echo [inventory] >> /etc/ansible/ansible.cfg
26 sed -i '/\[inventory\]/a\enable_plugins = aws_ec2' /etc/ansible/ansible.cfg
27 sed -i '/\[defaults\]/a\inventory = /home/ubuntu/aws_ec2.yaml' /etc/ansible/ansible.cfg
28 sed -i '/aws_ec2.yaml/a\remote_user = ubuntu' /etc/ansible/ansible.cfg
29 sed -i '/remote_user = ubuntu/a\private_key_file = /home/ubuntu/terraform-new-key1.pem' /etc/ansible/ansible.cfg
30
```

- list aws instance and tages to group

ansible-inventory --graph

ansible-inventory -i /opt/ansible/inventory/aws_ec2.yaml --list

```
aws_instance.e3_bastion_host (remote-exec): }
aws_instance.e3_bastion_host (remote-exec): @all:
aws_instance.e3_bastion_host (remote-exec): |--ungrouped:
aws_instance.e3_bastion_host (remote-exec): |--aws_ec2:
aws_instance.e3_bastion_host (remote-exec): | |--ip-10-0-3-58.ap-southeast-2.compute.internal
aws_instance.e3_bastion_host (remote-exec): | |--ec2-13-211-146-132.ap-southeast-2.compute.amazonaws.com
aws_instance.e3_bastion_host (remote-exec): |--@tag_Name_e3_redis:
aws_instance.e3_bastion_host (remote-exec): | |--ip-10-0-3-58.ap-southeast-2.compute.internal
aws_instance.e3_bastion_host (remote-exec): |--@aws_region_ap_southeast_2:
aws_instance.e3_bastion_host (remote-exec): | |--ip-10-0-3-58.ap-southeast-2.compute.internal
aws_instance.e3_bastion_host (remote-exec): | |--ec2-13-211-146-132.ap-southeast-2.compute.amazonaws.com
aws_instance.e3_bastion_host (remote-exec): |--@tag_Project_e3:
aws_instance.e3_bastion_host (remote-exec): | |--ip-10-0-3-58.ap-southeast-2.compute.internal
aws_instance.e3_bastion_host (remote-exec): |--@tag_Name_e3_easy_bastion_host:
aws_instance.e3_bastion_host (remote-exec): | |--ec2-13-211-146-132.ap-southeast-2.compute.amazonaws.com
aws_instance.e3_bastion_host (remote-exec): |--@tag_Project_e3_easy:
aws_instance.e3_bastion_host (remote-exec): | |--ec2-13-211-146-132.ap-southeast-2.compute.amazonaws.com
aws_instance.e3_bastion_host: Creation complete after 3m7s [id=i-0c011d6f59c7e4ba8]
Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
```

- **E5: depends on + output + local-exec + backend**

1. use depends on to meet condition during terraform resource creation

eg: need backend server ready to create bastion host

```

56   provisioner "local-exec" {
57     command = "echo aws_instance.e3_bastion_host.private_ip >> ip.txt"
58   }
59   depends_on = [aws_instance.e3_backend_server_1]
60 }

```

2. use output to show info after terraform creation # in the end of code
eg show public and private IP for both bastion host and backend server

```

79   output "bastion-host-IP1" {
80     value = aws_instance.e3_bastion_host.public_ip
81   }
82   output "bastion-host-IP2" {
83     value = aws_instance.e3_bastion_host.private_ip
84   }
85
86   output "e3_backend_server_1-IPs" {
87     value = aws_instance.e3_backend_server_1.private_ip
88   }

```

3. use local-exec to run command locally

```

78   provisioner "local-exec" {
79     command = <<-EOT
80     echo "${aws_instance.e3_backend_server_1.private_ip}" > ip.txt
81     EOT
82   }

```

4. use backend to store terraform state in remote location S3

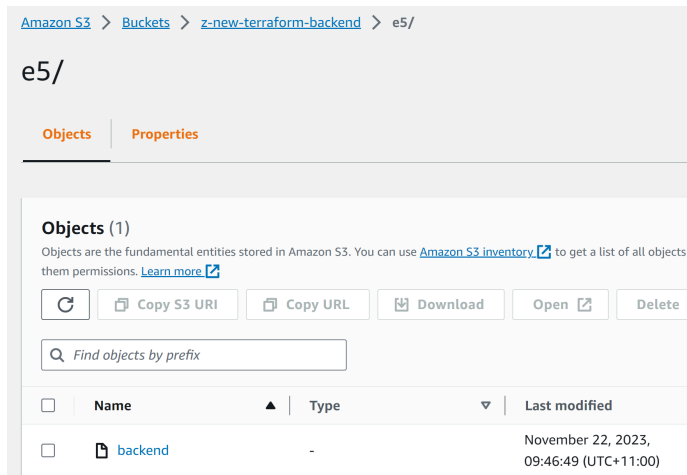
- create S3 bucket # ["z-new-terraform-backend"](#)
- create folder under bucket # "e5/"
- mention in backend.tf with key = "e5/backend"

```

backend.tf > terraform
1   terraform {
2     backend "s3" {
3       bucket = "z-new-terraform-backend"
4       key    = "e5/backend"
5       region = "ap-southeast-2"
6     }
7   }

```

- terraform will create state file "backend" in S3 at 1st time
terraform will call and query this backend file everytime and warn if state inconsistent



Error: Backend configuration changed

A change in the backend configuration has been detected, which may require migrating existing state.

If you wish to attempt automatic migration of the state, use "terraform init -migrate-state".

If you wish to store the current configuration with no changes to the state, use "terraform init -reconfigure".

- E6: Terraform module - VPC

1. search modules from Terraform Registry
2. use var to define common terms like "VPCname, CIDR zone"
 - take time for initial declare variables
 - but make code reusable and save time later
 - use same code just change value of vars for different env or projects
3. in vpc module
 - RT will be created automatically and associated with subnet accordingly
 - enable single NATgateway to save cost
 - enable auto assign public ip at launch for public subnet

```
map_public_ip_on_launch = true
```

- give VPC tags
- 4. create SG based on VPC, find output from terraform registry doc
 - after VPC created, need output of vpc id to create security group
 - use module.vpc.vpc_id (from doc output)

```
1 resource "aws_security_group" "allow_tls" {
2   name      = "e6-sg"
3   description = "Allow TLS inbound traffic"
4   vpc_id    = module.vpc.vpc_id
5 }
```

- use my ip

```
32 cidr_blocks = [var.MY_IP]
```

- refer to other sg

```
security_groups = [aws_security_group.bastion-test.id]
```

- update a sg by add rules
add new resource "aws_security_group_rule" to allow all servers within this sg to communicate to each other

```
resource "aws_security_group_rule" "sg-allow-all servers within SG to talk" {  
  type           = "ingress"  
  from_port      = 0  
  to_port        = 65535  
  protocol       = "tcp"  
  security_group_id = aws_security_group.backend-test.id  
  source_security_group_id = aws_security_group.backend-test.id  
}
```

- **E7: Terraform mudole - ELB + ASG + AMI**

1. launch instance from a VPC with subnet ID and security group, with user data to install web
2. create AMI based on above instance
3. create ELB and ASG based on above AMI and launch template
[https://terraformguru.com/terraform-real-world-on-aws-ec2/15-Autoscaling-with-Laun ch-Templates/](https://terraformguru.com/terraform-real-world-on-aws-ec2/15-Autoscaling-with-Laun-ch-Templates/)
4. create local rsa key
 - ssh-keygen -t rsa -C "e7-my-key" -f ~/.ssh/e7-my-key
 - aws ec2 import-key-pair --key-name "e7-my-key" --public-key-material fileb://~/.ssh/e7-my-key.pub
 - ssh-keygen -f e7-my-key -m PEM -e > e7-my-key.pem
 - use e7-my-key.pem to connect to ec2
5. create route53 public hosted zone, create record point load balancer
adjust records in godaddy
<https://mxtoolbox.com/> Domain lookup
6. ASG with dynamic scaling policy
 - set cloud watch alarm networkin & networkout metrics
 - Target tracking scaling with average network in as metric with target value
 - group size with desired capacity and min and max desired capacity

```
resource "aws_cloudwatch_metric_alarm" "network_alarm" {  
  alarm_name      = "network-traffic-high"  
  comparison_operator = "GreaterThanOrEqualToThreshold"  
  evaluation_periods = 2  
  metric_name     = "NetworkOut"  
  namespace      = "AWS/EC2"  
  period         = 300 # 5 minutes period  
  statistic       = "Average"  
  threshold       = 1000000 # Adjust the threshold based on your requirements  
  alarm_description = "Network traffic high, trigger scale-out policy"  
  alarm_actions    = [aws_autoscaling_policy.scale_out.arn]  
}
```


Create dynamic scaling policy

Policy type

Target tracking scaling

Scaling policy name

Target Tracking Policy

Metric type [Info](#)

Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider better scaling performance.

Average network in (bytes)

Target value

5000

Instance warmup [Info](#)

300

seconds

☐ Disable scale in to create only a scale-out policy

Group size



Specify the size of the Auto Scaling group by changing the desired capacity. You can also specify minimum and maximum scaling limits.

Desired capacity type

Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

Desired capacity

Specify your group size.

2

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

1

Equal or less than
desired capacity

Max desired capacity

3

Equal or greater than
desired capacity

Cancel

Update

In the context of AWS CloudWatch, "NetworkIn" and "NetworkOut" metrics are used to measure the network traffic of Amazon EC2 instances. These metrics provide insights into the incoming (inbound) and outgoing (outbound) network traffic for your instances.

• **NetworkIn (Inbound):**

- Measures the number of bytes received by the instance from all network interfaces. It indicates the amount of data that is coming into the instance.

• **NetworkOut (Outbound):**

- Measures the number of bytes sent out on all network interfaces by the instance. It indicates the amount of data that is leaving the instance.

For a website under peak traffic from massive users, you would typically see an increase in both "NetworkIn" and "NetworkOut" metrics. Here's what each might signify during peak traffic:

• **Peak NetworkIn:**

- Indicates a high volume of data being received by your EC2 instances. This could be a result of users accessing content, making requests, or uploading data to your application.

• **Peak NetworkOut:**

- Indicates a high volume of data being sent out from your EC2 instances. This could be a result of serving content, responding to user requests, or sending data to external services.

7. 11

- **E8: Terraform rds-mysql in backend**

1. create VPC with module, enable public IP assign
2. create 2 sg for both bastion host and rds instance (allow 3306 from bastion to rds)
3. create bastion host and rds instance (set mysql user and passwd)
4. ssh to bastion host and install mysql client

```
# sudo apt-get update && sudo apt install mysql-client-core-8.0
```

```
# mysql -h rds-endpoint -u your-mysql-username --password=your-db-passwd
```

- **E9: Terraform beanstalk blue/green**

1. create vpc
2. create sg for bastion, bean-elb, bean-sg, bean-elb2, bean-sg2
3. create bean1 (app + env) as blue, bean2 (app_env) as green
4. create setting for asg, elb
5. using join function to list more subnets

```
setting {
  namespace = "aws:ec2:vpc"
  name      = "ELBSubnets"
  value     = join(",", [module.vpc.public_subnets[0], module.vpc.public_subnets[1]])
}
```

6. create S3 and upload new version .war, create application version to user bucket and key

7. from bastion to each beanstalk app, validate web app from

```
/usr/share/tomcat{X}/ROOT
```

